

Triggers

A trigger is a stored program that is associated with a particular table and defined to activate for INSERT, DELETE, or UPDATE statements for that table. A trigger can be set to activate either before or after each row processed by the statement. The trigger definition includes a statement that executes when the trigger activates.

To create a trigger, use the `CREATE TRIGGER` statement. The definition indicates the particular type of statement for which the trigger activates (INSERT, UPDATE, or DELETE), and whether it activates BEFORE or AFTER rows are modified.

The basic syntax for trigger creation looks like this:

```
CREATE TRIGGER trigger_name          # the trigger name
    (AFTER | BEFORE)                 # when the trigger activates
    (INSERT | UPDATE | DELETE)       # what statement activates it
    ON table_name                     # the associated table
    FOR EACH ROW                      # what the trigger does
    BEGIN
        --variable declarations
        --trigger code
    END;
```

Create data a database triggers_WS and use it.

Example:

```
CREATE TABLE emp_work_hours (
    name varchar(45) NOT NULL,
    occupation varchar(35) NOT NULL,
    working_date date,
    working_hours varchar(10)
);
```

```
INSERT INTO emp_work_hours VALUES
('Robin', 'Scientist', '2020-10-04', 12),
('Warner', 'Engineer', '2020-10-04', 10),
('Peter', 'Actor', '2020-10-04', 13),
('Marco', 'Doctor', '2020-10-04', 14),
('Brayden', 'Teacher', '2020-10-04', 12),
('Antonio', 'Business', '2020-10-04', 11);
```

Trigger: BEFORE – INSERT

A BEFORE INSERT trigger runs just before a new row is added to a table. it's used to validate input data, modify values before saving, and enforce business rules. You can change the values using NEW.column before they are stored.

Example:

Write a trigger to prevent inserting negative working hours (set the working hours to 0 in case of a negative value):

```
DELIMITER //

CREATE TRIGGER bi_no_negative_hours
BEFORE INSERT ON emp_work_hours
FOR EACH ROW
BEGIN
IF NEW.working_hours < 0 THEN
SET NEW.working_hours = 0;
END IF;
END; //

DELIMITER ;

INSERT INTO emp_work_hours VALUES
('Markus', 'Former', '2020-10-08', 14);
```

```
INSERT INTO emp_work_hours VALUES  
( 'Alexander', 'Actor', '2020-10-012', -13);
```

Write a trigger to change the occupation from 'Scientist' to 'Doctor'.

```
DELIMITER //  
Create Trigger before_insert_occupation  
BEFORE INSERT ON emp_work_hours FOR EACH ROW  
BEGIN  
IF NEW.occupation = 'Scientist' THEN SET NEW.occupation =  
'Doctor';  
END IF;  
END //  
DELIMITER ;
```

```
INSERT INTO emp_work_hours VALUES ( 'Markus', 'Scientist', '2020-  
10-08', 14);
```

```
INSERT INTO emp_work_hours VALUES ( 'Alexander', 'Actor', '2020-  
10-12', 13);
```

Write separate triggers for each of the following:

1. To set the default working date to the current date if it is null.
2. To restrict the maximum working hour limit to 16 hours.
3. To set the occupation to a default value in case of a null value.

Trigger: After – INSERT

An AFTER INSERT trigger runs immediately after a new row is successfully inserted. It's used for logging activities, updating summary tables, and sending alerts. The data is already saved, so you cannot modify it, only react to it.

Example:

Write a trigger to log insert action into the work_log table.

Step 1: create the work_log table.

```
CREATE TABLE work_log (name varchar(25), action varchar(100));
```

Step 2: write trigger

```
CREATE TRIGGER ai_log_insert
AFTER INSERT ON emp_work_hours
FOR EACH ROW
BEGIN
INSERT INTO work_log(name, action)
VALUES (NEW.name, 'Inserted');
END;
```

Write a trigger to alert for long working hours

Step 1:

```
create table alerts (name varchar(25), message varchar(100));
```

Step 2:

```
CREATE TRIGGER ai_high_hours_alert
AFTER INSERT ON emp_work_hours
FOR EACH ROW
BEGIN
    IF NEW.working_hours > 12 THEN
        INSERT INTO alerts(message)
        VALUES ('High working hours recorded');
    END IF;
END;
```

Write separate triggers for each of the following:

1. To insert the record into a backup table. (Hint: Create the table appropriately)
2. To keep an occupation log file.

(Hint: Create the necessary tables appropriately)

Trigger: BEFORE – UPDATE

A BEFORE UPDATE trigger runs before an existing row is modified. It's used to validate updates, prevent illegal changes, and adjust values before saving. You can access using OLD.column for the existing value and NEW.column for the updated value. And you can modify NEW values before they are saved.

Example:

Write a trigger to ensure the working hour is a positive value before updating.

```
CREATE TRIGGER bu_no_negative_hours
BEFORE UPDATE ON emp_work_hours
FOR EACH ROW
BEGIN
IF NEW.working_hours < 0 THEN
SET NEW.working_hours = 0;
END IF;
END;
```

Write a trigger to prevent name change before update.

```
CREATE TRIGGER bu_prevent_name_change
BEFORE UPDATE ON emp_work_hours
FOR EACH ROW
BEGIN
IF OLD.name <> NEW.name THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Name cannot be changed';
END IF;
END;
```

Write separate triggers for each of the following:

1. To limit the maximum working hours to 16 before the update.
2. To ensure the working date is not a future date before the update.

Trigger: AFTER – UPDATE

An AFTER UPDATE trigger runs after a row has been updated successfully. It's used track changes (audit logs), trigger notifications, and maintain derived data. You cannot change the updated data, only observe and act on it.

Example:

Write a trigger to maintain the hours_log table to record the updated value of working hours. (Hint: Create the hours_log table)

```
CREATE TRIGGER au_track_hours_change
AFTER UPDATE ON emp_work_hours
FOR EACH ROW
BEGIN
  IF OLD.working_hours <> NEW.working_hours THEN
    INSERT INTO hours_log(name, old_hours, new_hours)
    VALUES (NEW.name, OLD.working_hours, NEW.working_hours);
  END IF;
END;
```

Write separate triggers for each of the following:

1. To maintain an updated log table with name and action attribute to keep a record of all the updates. (Hint: use the work_log table)
2. To keep an alert message table for large deviations (more than 5 hours) in the working hours update. (Hint: use the alert table)
3. To track the occupation changes in a log file. (Hint: use the work_log table.)

Trigger: BEFORE – DELETE

A BEFORE DELETE trigger runs before a row is removed from the table. It's used to prevent deletion under conditions, backup data before deletion, and enforce constraints. You can cancel the deletion if rules are violated.

Example:

Write a trigger to prevent the deletion of working hours of more than 12 hours.

```
CREATE TRIGGER bd_protect_high_hours
BEFORE DELETE ON emp_work_hours
FOR EACH ROW
BEGIN
IF OLD.working_hours > 12 THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Cannot delete high work hour record';
END IF;
END;
```

Write a trigger to back up before deleting. (Hint: Create the backup table)

```
CREATE TRIGGER bd_backup
BEFORE DELETE ON emp_work_hours
FOR EACH ROW
BEGIN
INSERT INTO emp_backup SELECT * FROM emp_work_hours WHERE name =
OLD.name;
END;
```

Write separate triggers for each of the following:

1. To keep a log of the delete attempt. (Hint: create a delete_log table with name and action attributes.)
2. To prevent the deletion of the record with the occupation 'Doctor'.
3. To validate that the record exists.

Trigger: AFTER – DELETE

An AFTER DELETE trigger runs after a row has been deleted. It's used to maintain logs, archive deleted data, and update counts/statistics. Only OLD values are available (because the row is already gone).

Example:

Write a trigger to log the deletion of records. (Hint: use the work_log table)

```
CREATE TRIGGER ad_log_delete
AFTER DELETE ON emp_work_hours
FOR EACH ROW
BEGIN
INSERT INTO work_log(name, action)
VALUES (OLD.name, 'Deleted');
END;
```

Write separate triggers for each of the following:

1. To archive the deleted records.
2. To track deletion history by keeping track of the deleted time.
3. To alert about the deletion action. (Hint: use the alerts table)